You have to list what skills I need and the depth of these skills to complete this project sprint

## Sprint 2 — PostgreSQL + Data Model

**Goal:** Learn how your application stores persistent state.

Start designing the database.

Initially create:

users

Show more

For this sprint, you **do not need to become a PostgreSQL expert**. You need enough database knowledge to understand persistence, design a sound relational model, implement it, and expose one working API path.

I'd break the required skills into **4 levels of depth**:

| Skill | Depth needed | What you should be able to do |
|---|---|---|
| **1. Relational database fundamentals** | 🟢 Intermediate | Understand tables, rows, columns, primary keys, foreign keys, relationships, constraints, indexes |
| **2. PostgreSQL** | 🟢 Intermediate | Create tables, choose PostgreSQL data types, write CRUD queries, use timestamps, constraints, indexes, transactions |
| **3. Database/schema design** | 🟡 Strong intermediate | Translate `Document` and `DocumentVersion` into a normalized schema and correctly model 1-to-many relationships |
| **4. Application ↔ database integration** | 🟡 Strong intermediate | Connect your application to PostgreSQL, use migrations/ORM or SQL, create/retrieve documents through an API |

### 1. Relational database fundamentals — Intermediate

You should understand:

- Tables vs. objects/entities
- Rows and columns
- **Primary keys**
- **Foreign keys**
- One-to-one vs. one-to-many relationships
- `NULL`
- `UNIQUE`, `NOT NULL`, `CHECK`
- Basic normalization
- Why relational integrity matters

For this sprint, you should be able to look at:

`documents → document_versions`

and immediately understand:

> One document can have many versions, and every version belongs to exactly one document.

You **don't need deep database theory** or advanced normalization proofs.

---

### 2. PostgreSQL — Intermediate

You should be comfortable with SQL such as:

SQL

```sql
CREATE TABLE ...
INSERT INTO ...
SELECT ...
UPDATE ...
DELETE ...
```

And especially:

- `CREATE TABLE`
- `ALTER TABLE`

- `SELECT ... WHERE`
- `INSERT ... RETURNING`
- `JOIN`
- `ORDER BY`
- `LIMIT`
- Basic aggregation
- UUIDs
- `TEXT` / `VARCHAR`
- `TIMESTAMP` / `TIMESTAMPTZ`
- Boolean fields
- Indexes
- Foreign-key constraints
- Unique constraints
- Transactions

You should also understand **why** you're choosing a particular PostgreSQL type rather than merely copying a schema.

For example, you should be able to reason about:

```
id
tenant_id
created_at
updated_at
version_number
content_hash
status
```

rather than treating every field as simply `VARCHAR`.

**Depth target:** You should be able to write the initial schema yourself, with documentation available when you forget exact PostgreSQL syntax.

---

### 3. Database/schema design — Strong Intermediate

This is probably the **most important skill in Sprint 2**.

You need to translate the application's concepts into relational structures.

At minimum, you should be able to design:

```
users
documents
document_versions
conversations
messages
```

and identify relationships such as:

```
tenant
  └── users

tenant
  └── documents
       └── document_versions

conversation
  └── messages
```

You should understand why `document_versions` is a separate table rather than putting something like:

```
version_1
version_2
version_3
```

inside `documents`.

You also need to understand the purpose of:

- `document_id` as a foreign key
- `version_number`
- `content_hash`
- `storage_key`
- `embedding_model_version`
- lifecycle `status`
- timestamps

### Particularly important: versioning

Your architecture explicitly requires versions and hashes, so you should understand this lifecycle:

```
Document
   │
   ├── Version 1
   │      └── content_hash = ABC
   │
   ├── Version 2
   │      └── content_hash = DEF
   │
   └── Version 3
          └── content_hash = XYZ
```

You should be able to explain:

> What changes when a document is updated, what remains stable, and how the system detects whether content actually changed.

You **do not yet need** advanced event sourcing, temporal databases, CQRS, or distributed database architecture.

---

### 4. Application ↔ PostgreSQL integration — Strong Intermediate

This is what turns the database exercise into an application sprint.

You need to know how your backend talks to PostgreSQL.

Depending on your stack, learn the equivalent of:

```
Application
    ↓
Database driver / ORM
    ↓
PostgreSQL
```

You should understand:

- Database connection configuration
- Connection pooling at a basic level
- Migrations
- Models/entities
- CRUD operations
- Transactions
- Mapping database records → API responses

- Basic error handling
- Foreign-key failures
- Unique-constraint failures

You should also understand **parameterized queries** and why you shouldn't construct SQL by concatenating user input.

---

## The actual depth target

I'd use this scale:

**Level 1 — Awareness**

> "I know what a primary key is."

Not enough for this sprint.

**Level 2 — Guided implementation**

> "I can build this if I follow a tutorial."

Almost enough, but you'll struggle when the schema needs architectural decisions.

**Level 3 — Independent implementation**

> "I can design these tables, implement them, and debug normal problems, using documentation when necessary."

**This is your target for Sprint 2.**

**Level 4 — Expert**

> "I can optimize query plans, design partitioning strategies, tune PostgreSQL internals, handle replication, etc."

Not necessary.

---

# Your Sprint 2 skill checklist

### Must know before/during the sprint

- [ ] Relational database concepts
- [ ] Primary/foreign keys
- [ ] One-to-many relationships
- [ ] SQL CRUD
- [ ] PostgreSQL data types
- [ ] `NOT NULL`, `UNIQUE`, `CHECK`
- [ ] Foreign-key constraints
- [ ] Index basics
- [ ] UUIDs
- [ ] Timestamps
- [ ] Transactions
- [ ] Database migrations
- [ ] ORM/driver basics for your application stack
- [ ] Schema normalization
- [ ] Document/version data modeling
- [ ] Content hashes and change detection
- [ ] API ↔ database integration

### Nice to know, but not required

- [ ] Query-plan analysis (`EXPLAIN`)
- [ ] Advanced indexing
- [ ] PostgreSQL JSONB
- [ ] CTEs
- [ ] Window functions
- [ ] Database views
- [ ] Stored procedures
- [ ] Triggers
- [ ] Replication
- [ ] Partitioning
- [ ] PostgreSQL performance tuning

---

## What "done" should prove

Your sprint shouldn't end with just five tables existing.

You should be able to demonstrate this flow:

```
POST /documents
       |
       ▼
Application validates input
       |
       ▼
PostgreSQL INSERT
       |
       ▼
Document created
       |
       ▼
document_id returned
       |
       ▼
GET /documents/{id}
       |
       ▼
PostgreSQL SELECT
       |
       ▼
Document returned
```

And you should be able to explain how that document relates to its future versions.

**If you can design the schema, create it through migrations, insert a document, retrieve it through the API, and explain the document/version relationship, you're at the right skill depth for Sprint 2.**

Are these skills required to get a junior AI Developer job or I can skip these now

Yes—but **you can skip going deep into them right now** if your goal is a **Junior AI Developer** role.

The key distinction is:

> **You need database literacy, not database specialization.**

For a junior AI Developer, I'd prioritize your skills like this:

| Skill | Junior AI Dev importance | What you need |
|---|---|---|
| Python | 🔴 Essential | Strong |
| APIs / backend basics | 🔴 Essential | Strong |
| LLM APIs & AI application development | 🔴 Essential | Strong |
| RAG / embeddings / vector search | 🔴 Essential | Strong |
| Git | 🔴 Essential | Comfortable |
| SQL | 🟠 Important | Intermediate |
| PostgreSQL | 🟠 Important | Basic–Intermediate |
| Data modeling | 🟠 Important | Basic–Intermediate |
| Docker | 🟠 Important | Basic–Intermediate |
| Cloud/deployment | 🟠 Important | Basic |
| Frontend | 🟡 Useful | Basic |
| Advanced PostgreSQL | ⚪ Not necessary | Skip for now |

## So for your current Sprint 2

**Don't skip it completely. Reduce the depth.**

You don't need to learn PostgreSQL like a backend/database engineer.

Learn enough to confidently do:

```
Python application
       ↓
API
       ↓
PostgreSQL
       ↓
```

```
documents
document_versions
conversations
messages
```

Specifically, I'd make your Sprint 2 target:

**SQL → Intermediate**
**PostgreSQL → Basic/Intermediate**
**Data modeling → Intermediate**
**Database internals → Skip**

You should know what a primary key, foreign key, index, transaction, migration, and one-to-many relationship are. You should be able to write basic SQL and connect your application to PostgreSQL.

You **do not** need to spend weeks learning query optimization, replication, partitioning, database administration, etc.

### Why it matters for AI jobs

Modern AI development isn't just:

```
LLM → answer
```

Real AI applications often look more like:

```
User
 ↓
API
 ↓
Application
 ├── PostgreSQL → users/documents/conversations
 ├── Object storage → files
 ├── Vector DB → embeddings
 └── LLM → generation
```

So a junior AI developer who can build the **whole pipeline** is considerably more useful than someone who only knows how to call an LLM API.

### My recommendation for you

**Don't skip Sprint 2. Compress it.**

Aim for **3–5 days of focused learning/building**, not mastery.

Your progression should roughly be:

**Python → APIs → SQL/PostgreSQL → data modeling → embeddings → RAG → LLM applications → deployment**

For the job market, I'd put **much more of your learning time into AI application development, RAG, evaluation, agents/tool use, Python/backend, and deployment** than advanced PostgreSQL.

So: **learn PostgreSQL enough to build this sprint, then move on.**